

Assignment 3

Daniel Göransson(MPDSC, dangora@chalmers.se),
Jonatan Markusson(MPALG, jonmarku@chalmers.se)

May 4, 2026

Group 26

Problem 1

a)

```
def mapper(index, row):
    cols = split data by (',')

    star = col containing star
    constellation = col containing constellation
    mineral_value = col containing mineral value

    if star == "Prime"
        self.by = constellation
    else:
        star_system = self.Star + " " + constellation

    yield (star_system, mineral_value)

def reducer(star_system, list_of_mineral_values)
    yield (star_system, sum(list_of_mineral_values))
```

b)

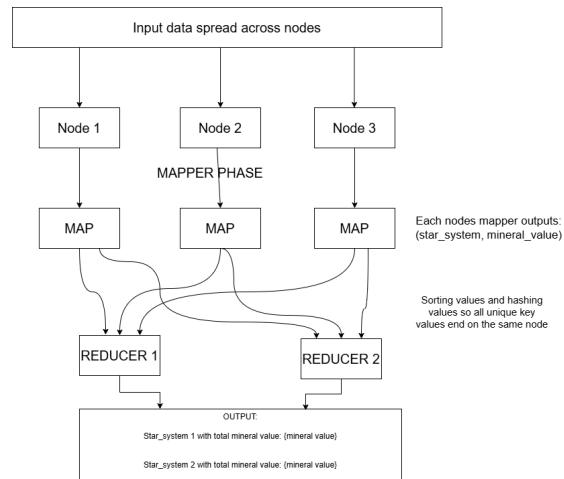


Figure 1: Diagram

c)

Star system	Total value
Capella	1248
Alpha Cancri	2082
Gamma Sagittae	2899
Beta Lyrae	2431
Alpha Geminorum	4257

Table 1: Total mineral values for these star systems

Problem 2

a)

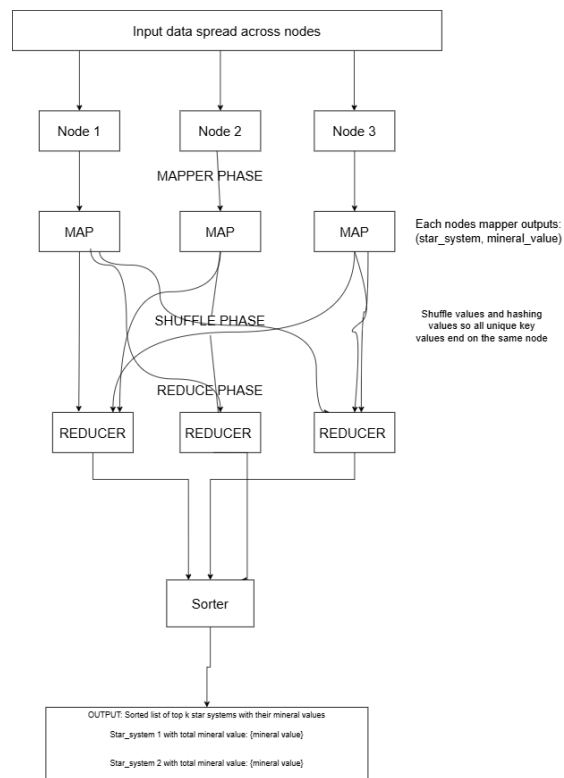


Figure 2: Top k diagram

b)

Planet	Value
Scorpii Beta	12680
Tauri Delta	12080
Ceti Alpha	11739
Centauri Alpha	11476
Aurigae Delta	11005
Cephei Beta	10447
Zeeman Prime	10421
Vulpeculae Zeta	9404
Normae Beta	9153
Brahe Delta	9100

Table 2: Top 10 planet values

1 Problem 3

1.1 a)

Pseudo code for the given task:

```
class twitter_MR():  
    def map(section of data):  
  
        lines = read the line  
        user_id, following_string = split lines by (":")
```

```

if not following_string:
    zero_following_flag = 1
    follow_count = 0
else:
    follow_count = length(split by spaces on follower string)
    zero_following_flag = 0

yield("GLOBAL", (user_id, follow_count, follow_count, +1
    zero_following_flag))

def combiner(list of tuples):

    sum_followers = 0
    max_follower_count = 0
    max_follower_id = 0
    total_user_count = 0
    sum_zero_following = 0

    for tuple in list of tuples:
        sum_followers = sum_followers + tuple.follow_count
        total_user_count += 1
        sum_zero_following += tuple.zero_following_flag

        if tuple.follow_count > max_follower_count:
            max_follower_count = tuple.follow_count
            max_follower_id = tuple.user_id

    yield("GLOBAL", (max_follower_id, max_follower_count, sum_followers,
        total_user_count, sum_zero_following))

def reducer(list of tuples):
    global_max_id = 0
    global_max_count = 0
    global_follower_total = 0
    global_total_users = 0
    global_zero_following = 0

    for tuple in tuple list:
        global_follower_total = global_follower_total + tuple.
            sum_followers
        global_total_users = global_total_users + tuple.total_user_count
        global_zero_following = global_zero_following + tuple.
            sum_zero_following

        if tuple.max_follower_count > global_max_count:
            global_max_count = tuple.max_follower_count
            global_max_id = tuple.max_follower_id

    average_follow_count = global_follower_total / global_total_users

    yield("GLOBAL", (global_max_id, global_max_count,
        average_follow_count, global_zero_following))

```

Diagram showing the structure:

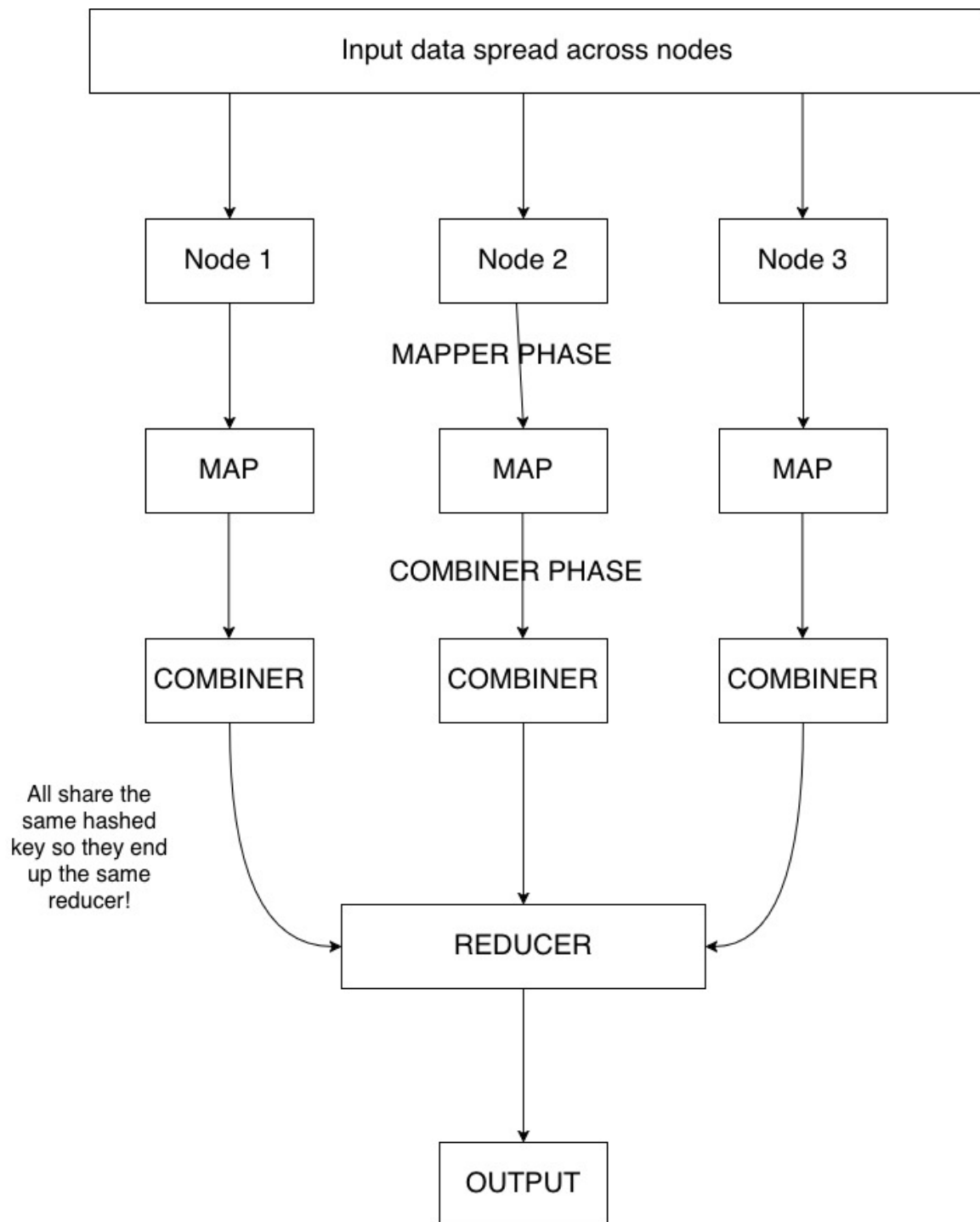


Figure 3: MapReduce architecture

c)

Single-core runtime: 40.746 seconds

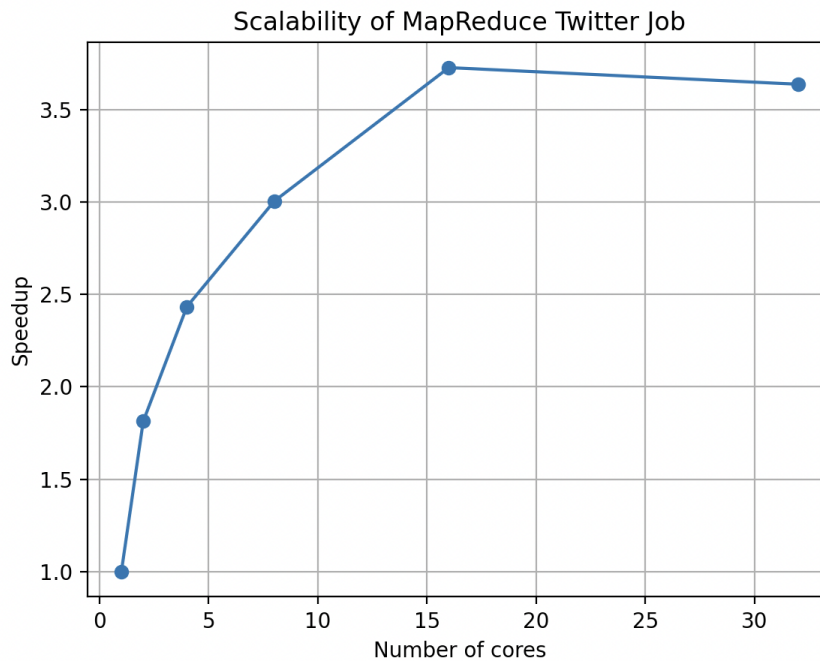


Figure 4: Plot showing speedup based on number of cores

d)

Here are the results obtained from running the MapReduce job on the full dataset:

- **Most followed ID:** 813286
- **Most followed (accounts followed):** 770155
- **Average followed:** 35.25
- **Count of accounts that follow no-one:** 5,963,082

2 Problem 4

2.1 a)

```
class TwitterFollowers(MRJob):

    def steps():
        return [MRStep(mapper = mapper1, combiner = reducer1, reducer =
            reducer1)
                MRStep(mapper = mapper2, combiner = combiner, reducer =
                    reducer2)]

    def mapper1(line):
        user_id, following_string = line.split by (":") and strip
            whitespaces

        yield(user_id, 0) # Ensure they are still counted

        if following_string is NOT empty:
            follower_list = split folling string by spaces
            for each user in follower_list:
                yield(user, 1)

    def reducer1(user, counts):
        total_followers = 0
```

```

        for each count in counts:
            total_followers += count

        yield(user, total_followers)

def mapper2(user, followers)
    if followers is 0:
        is_zero_followers = 1
    else:
        is_zero_followers = 0

    yield("GLOBAL", (user_id, followers, followers, 1, is_zero_followers
        ))

def combiner(_, list_of_tuples)
    local_max_id = -1
    local_max_count = -1
    sum_followers = 0
    total_users = 0
    sum_zeroes = 0

    for every tuple in list_of_tuples:
        sum_followers = sum_followers + tuple.followers
        total_users = total_users + tuple.Count_of_Users
        sum_zeros = sum_zeros + tuple.is_zero_followers

        if tuple.followers > local_max_count:
            local_max_count = tuple.followers
            local_max_id = tuple.user_id

    yield("GLOBAL", (local_max_id, local_max_count, sum_followers,
        total_users, sum_zeros))

def reducer2(_, list_of_tuples):
    global_max_id = -1
    global_max_count = -1
    global_total_followers = 0
    global_total_users = 0
    global_total_zeros = 0

    for every tuple_val in list_of_tuples:
        global_total_followers += tuple.sum_followers
        global_total_users += tuple.total_users
        global_total_zeros += tuple.sum_zeros

        if tuple.local_max_count > global_max_count:
            global_max_count = tuple.local_max_count
            global_max_id = tuple.local_max_id

    average_followers_count = global_total_followers /
        global_total_users

    yield(global_max_id, global_max_count, average_followers_count,
        global_total_zeros)

```

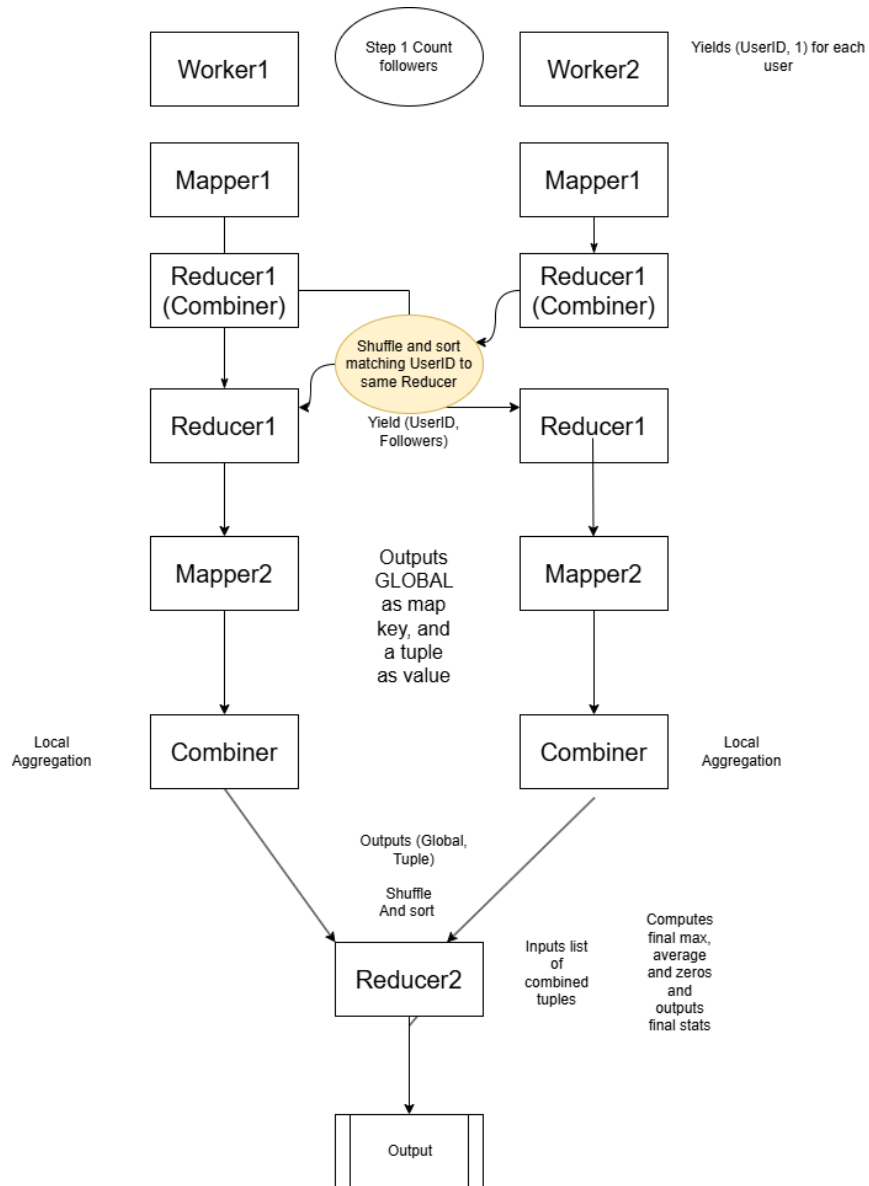


Figure 5: Data flow

c)

Single-core runtime on the dataset was: 318.138 seconds

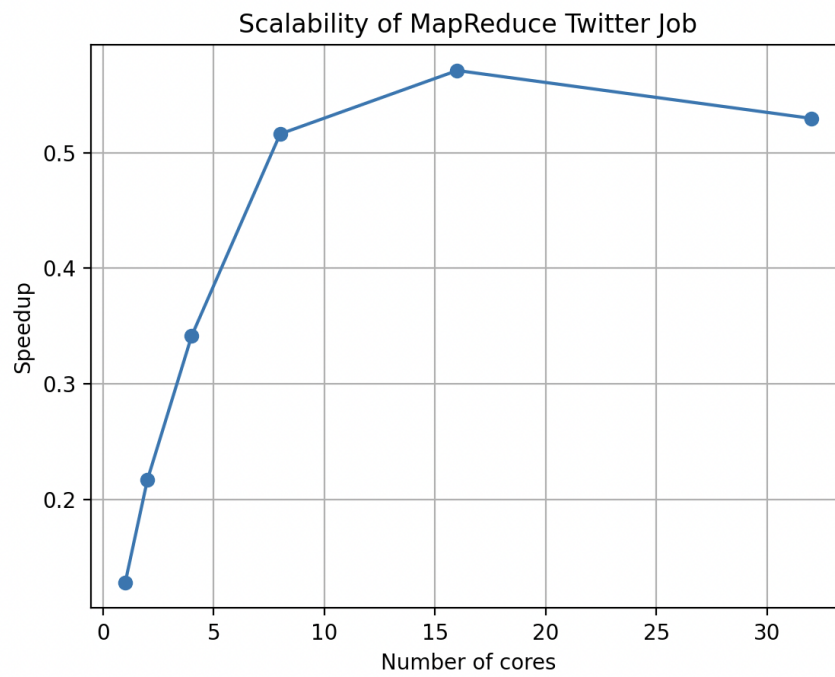


Figure 6: Plot showing speedup based on number of cores

d)

Results from the full dataset is:

Metric	Value
most followers id	19,058,681
most followers	2,997,469
average followers	35.252
count no followers	1,548,949